

CS 4610 HW 2

1. We know C is not type-safe and we have shown (see slides) how to manipulate the value of some variable without explicitly reference its name in the code. In fact, we can even manipulate the code to change the control flow without explicitly writing function calls or returns. Look at the following C code.

```
#include <stdio.h>
#include <stdlib.h>

void fun2(int a) {
    printf("a=%d\n", a);
    exit(0);
}

void fun1() {
    // create a pointer and point it to its own address on the stack
    int *p = (int *)&p;

    // save the original return address
    int orig_ret = p[3];

    // overwrite fun1 return address with the address of fun2
    p[3] = (int)fun2;

    // Set up fun2 argument
    p[5] = 7083;
}

int main() {
    fun1();
    return 0;
}
```

We see `main()` calls `fun1()`, which then calls `fun2(7083)`. The result is `"a=7083"`. Now your job is to change the code in `fun1()` such that it will run `fun2()` with argument 7083, but without actually writing `"fun2(7083)"` in the code or accepting any user input. Your code in `fun1()` is expected to be a set of pointer operations without any function call. (50pts)

Write and test your code. Submit your code, screen-shot and explain the results.

Hints: (1) You need to remember how `cdecl` works; (2) `objdump` helps a lot; (3) compile your code using gcc options `-O0` and `-m32`. `-O0` means no optimization, so we have a simple stack layout. `-m32` means forcing 32-bit code that by default uses the `cdecl` calling convention. (4) If your machine is 64-bit, you can try to install 32-bit gcc support using command `"apt-get install gcc-multilib"` in Ubuntu.

```

git > 4610 > C hw2_q1v2.c
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void fun2(int a) {
5      printf("a=%d\n", a);
6  }
7
8  void fun1() {
9      // create a pointer and point it to its own address on the stack
10     int *p = (int *)&p;
11
12     // save the original return address
13     int orig_ret = p[6];
14
15     // overwrite fun1 return address with the address of fun2
16     p[6] = (int)fun2;
17
18     // set the fake return address for fun2 (so it doesn't crash when it finishes)
19     p[7] = orig_ret;
20
21     // Set the argument for fun2
22     p[8] = 7083;
23 }
24
25 int main() {
26     fun1();
27     return 0;
28 }

```

PROBLEMS OUTPUT PORTS TERMINAL DEBUG CONSOLE

```

● targeted@DESKTOP-0DF22VO:~/git/4610$ make
gcc -m32 -O0 -o hw2_q1 hw2_q1v2.c
● targeted@DESKTOP-0DF22VO:~/git/4610$ ./hw2_q1
a=7083
○ targeted@DESKTOP-0DF22VO:~/git/4610$ 

```

By creating a local pointer `p` pointing to its own address, I was able to access the stack frame relative to `fun1`'s local variables. Because I compiled with `-m32` and `-O0`, the stack layout follows the standard 32-bit cdecl calling convention. Due to compiler alignment and local variable allocation, `p` was placed at `[ebp - 20]`. Using pointer arithmetic, I identified that `p[5]` was the Saved EBP and `p[6]` was the Return Address located at `[ebp + 4]`. I saved this original return

address to a variable and then overwrote p[6] with the memory address of fun2() to successfully hijack the control flow.

To pass the argument 7083 to fun2() and prevent a segmentation fault, I had to construct the stack frame that fun2() would expect to see upon waking up. When fun1() executes its ret instruction, it pops p[6] into the Instruction Pointer (EIP) and moves the Stack Pointer up by 4 bytes, so ESP now points to p[7]. I placed the original return address at p[7], and placed the required argument 7083 at p[8]. As a result, fun2() executes, prints the correct value, and safely returns to main() without crashing.

My code for hw2_q1.c

```
#include <stdio.h>
#include <stdlib.h>

void fun2(int a) {
    printf("a=%d\n", a);
}

void fun1() {
    // create a pointer and point it to its own address on the stack
    int *p = (int *)&p;

    // save the original return address
    int orig_ret = p[6];

    // overwrite fun1 return address with the address of fun2
    p[6] = (int)fun2;

    // set the fake return address for fun2 (so it doesn't crash when it
    finishes)
    p[7] = orig_ret;

    // Set the argument for fun2
    p[8] = 7083;
}
```

```
int main() {  
    fun1();  
    return 0;  
}
```

I had a really hard time with this one because the program just kept segfaulting. I had another version that utilized `exit 0`, but I think this one is better. If you want to see it, you can look at the previously submitted document (or I don't know what you can do on your end).

2. In class, we gave a very basic macro design to implement stack guard with a canary. In that design, we haven't talked about how to define `__SECRET_VAR` in the macros. A simple way to define `__SECRET_VAR` is some macro like

```
#define __SECRET_VAR 11223344
```

But the issue is that it is a constant, so an attacker can know. A better strategy is to make it a variable and assign a random number to it at run time.

Your job is to write the entire set of macros in a header file "mycanary.h" so a user's program can include the header file and use all macros for stack protection. You have already been given the following user's code, which uses four macros:

`DEF_CANARY()`, `PLACE_CANARY()`, `SAFE_RETURN()`, and `INIT_CANARY()`.

Please complete all four macros in the header file "mycanary.h" so `__SECRET_VAR` can be assigned randomly at run time. Submit the content of your "mycanary.h". Screen-shot the result to show your macros can successfully detect buffer overflow in the following code. (50pts)

```
#include<stdio.h>  
#include<string.h>  
#include"mycanary.h"  
DEF_CANARY(); /* define canary as global*/  
void vul_fun(char *str) {
```

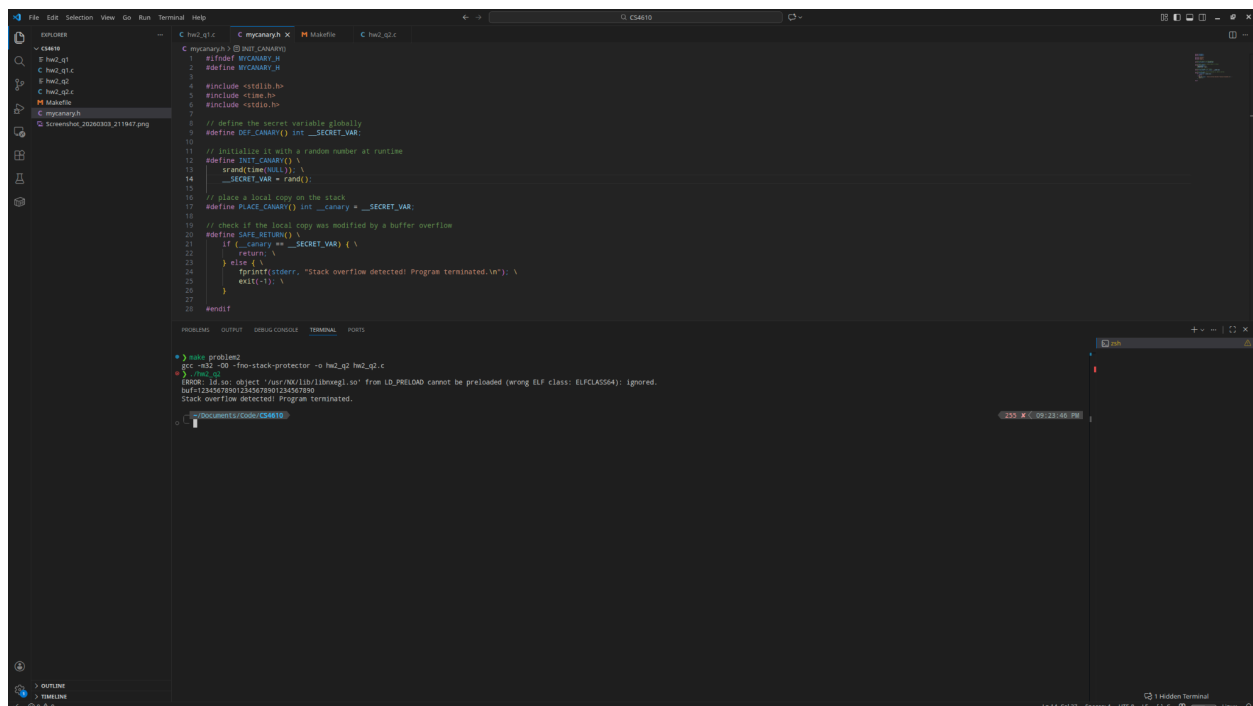
```

PLACE_CANARY(); /* place a canary */
char buf[16];
strcpy(buf, str);
printf("buf=%s\n", buf);
SAFE_RETURN(); /* check canary and return */
}

int main(){
    INIT_CANARY(); /* randomize the value of canary */
    vul_fun("123456789012345678901234567890");
}

```

Hints: You can define a global variable in DEF_CANARY, then assign a random value (using random functions in C) to __SECRET_VAR in INIT_CANARY. PLACE_CANARY() and SAFE_RETURN() can be found in slides.



My code for mycanary.h

```

#ifndef MYCANARY_H
#define MYCANARY_H

#include <stdlib.h>

```

```
#include <time.h>
#include <stdio.h>

// define the secret variable globally
#define DEF_CANARY() int __SECRET_VAR;

// initialize it with a random number at runtime
#define INIT_CANARY() \
    srand(time(NULL)); \
    __SECRET_VAR = rand();

// place a local copy on the stack
#define PLACE_CANARY() int __canary = __SECRET_VAR;

// check if the local copy was modified by a buffer overflow
#define SAFE_RETURN() \
    if (__canary == __SECRET_VAR) { \
        return; \
    } else { \
        fprintf(stderr, "Stack overflow detected! Program\n"); \
        terminated.\n"); \
        exit(-1); \
    }

#endif
```